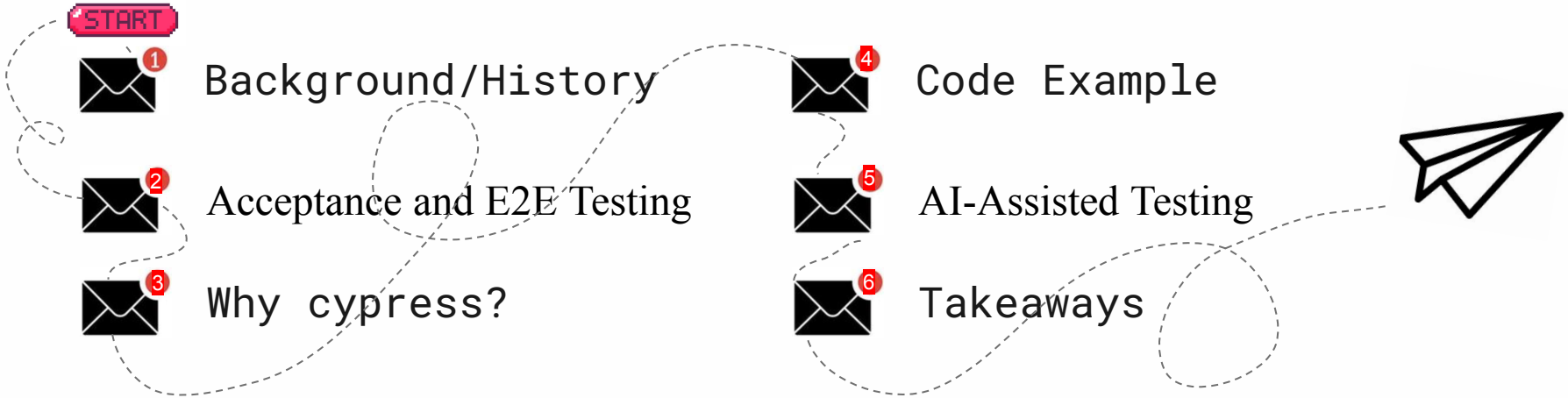
Automated Acceptance Testing with Cypress



By Jonny

Agenda


START
1
Background/History
2
Acceptance and E2E Testing
3
Why cypress?
4
Code Example
5
AI-Assisted Testing
6
Takeaways

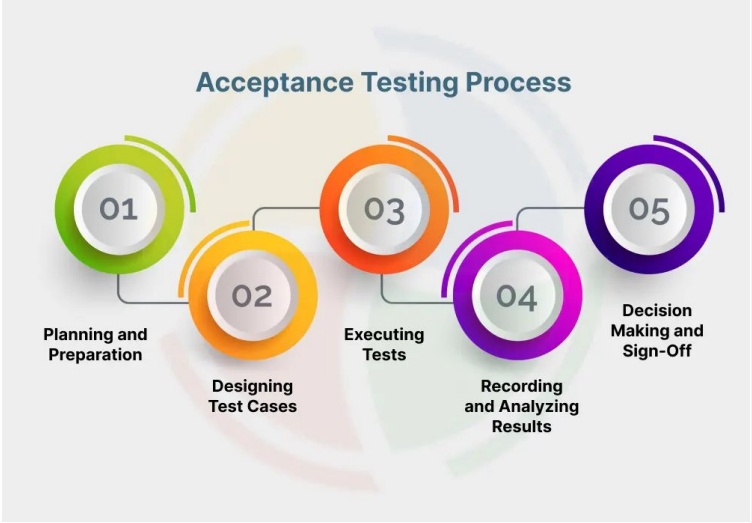
Background



- Cypress is a JavaScript-based testing framework for modern web applications.
- Released publicly in 2017 and developed by Cypress.io.
- Designed to make automated browser testing easier for developers.
- Supports end-to-end (E2E) and component testing.
- Provides interactive debugging and automatic waiting.

What is automated acceptance testing?

- Acceptance testing verifies that software meets specified requirements.
- Tests are based on expected user behavior and business requirements.
- Automated acceptance tests execute these checks without manual interaction.
- Cypress can automate acceptance scenarios through browser-based E2E tests.



Why cypress?

Automatic Waiting Retries queries and assertions to reduce timing related fails	**Easy Debugging** Interactive Test Runner that shows what's going on
Developer friendly Write in JS/TS or use plain English	**CI/CD Integration** Run tests automatically on push/pulls (ie GitHub Actions)

Similar alternatives you may know: Playwright, Selenium

SLIDESGO

Acceptance Testing Example

User story -> I want to log into my account so I can access my dashboard

Acceptance criteria:

- Valid credentials log user in
- Invalid credentials gives an error
- Login takes user to dashboard

Cypress!

group of tests

individual test

```
describe('Login Acceptance Test', () => {
    it('logs in with valid credentials', () => {
      cy.visit('/login')

      cy.get('[data-cy="email"]').type('test@example.com')
      cy.get('[data-cy="password"]').type('test123')
      cy.get('[data-cy="login"]').click()

      cy.url().should('include', '/dashboard')
    })
  })
```

AI features

- cy.prompt()

```
cy.prompt([
  'visit http://localhost:3000/login',
  'enter test@example.com in email',
  'enter test123 in password',
  'click the login button',
  'verify the dashboard is displayed'
])
```

same lines, natural language

- Self healing tests - AI tests automatically adapt to work when UI selectors change
- Studio AI - Record interactions, get assertion suggestions

Good use of cy.prompt

```
cy.prompt([
  'Given there is an inactive user on the page',
  'When the admin tries to deactivate the already inactive user',
  'But the user is already in inactive status',
  'Then the system should show a message that user is already inactive',
  'But the user status should remain unchanged',
])
```



```
it('Scenario: User cannot be deactivated if already inactive', () => {
  cy.visit('/user-management')

  // Prompt step 1: Given there is an inactive user on the page
  cy.get('#user-list .status-inactive').should('be.visible')

  // Prompt step 2: When the admin tries to deactivate the already inactive user
  cy.get('#user-list div:nth-child(3) > .user-actions > .btn-danger').click()

  // Prompt step 3: But the user is already in inactive status
  cy.get('#user-list .status-inactive').should('contain', 'inactive')

  // Prompt step 4: Then the system should show a warning message
  cy.get('.warning-notification').should('be.visible')

  // Prompt step 5: But the user status should remain unchanged
  cy.get('#user-list .status-inactive').should('contain', 'inactive')
})
)
```

Conclusion

- Cypress automates browser interactions to verify application behavior.
- Automated acceptance tests help ensure that software meets user requirements.
- Features like automatic waiting, interactive debugging, and AI assistance simplify test development.

Sources

https://www.cypress.io/#create

http://docs.cypress.io/app/get-started/why-cypress

https://docs.cypress.io/app/guides/cypress-studio

https://docs.cypress.io/api/commands/prompt